

Lab 3: Coding and Maximizing a Likelihood

SOCIOL 723 — Social Statistics II — Thursday, September 10

Wenhao Jiang

Table of contents

1	Goals	1
2	The likelihood as an R function	2
2.1	A binomial likelihood	2
2.2	Why we work in logs	2
3	Maximizing: the Poisson case	3
3.1	Standard errors from the Hessian	4
4	Two parameters: the normal likelihood	4
4.1	The likelihood surface	5
5	What non-identification looks like	6
6	Logit from scratch	7
7	The three tests, by hand	8
8	Exercises	10
9	Session information	10

1 Goals

1. Write a log-likelihood as an R function and plot it.
2. Maximize it numerically with `optim()`, and check the answer against the closed form.
3. Get standard errors from the Hessian, and check them against the analytic Fisher information.
4. See what non-identification looks like.
5. Code the logit likelihood from scratch and reproduce `glm()`.
6. Compute the likelihood ratio, Wald, and score tests by hand.

```
library(dplyr)
library(ggplot2)
gss <- readRDS("../Data/gss_earnings.rds")
```

2 The likelihood as an R function

2.1 A binomial likelihood

Ten draws, seven blue marbles. The likelihood is $\mathcal{L}(\theta) = \binom{10}{7}\theta^7(1-\theta)^3$.

```
lik <- function(theta) dbinom(x = 7, size = 10, prob = theta)
curve(lik, from = 0, to = 1, n = 400, col = "navy", lwd = 2,
      xlab = expression(theta), ylab = expression(L(theta)))
abline(v = 0.7, col = "firebrick", lty = 2)
```

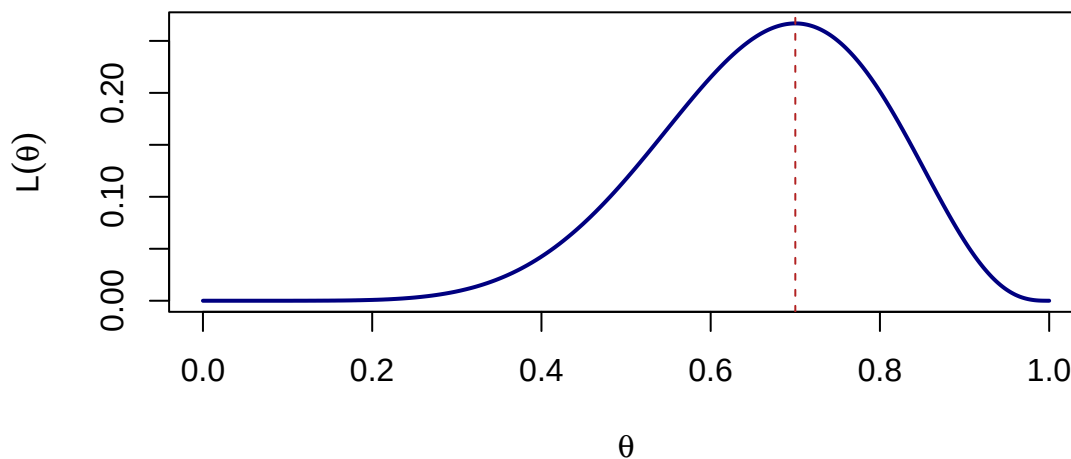


Figure 1: Binomial likelihood, 7 successes in 10 trials.

Note what `dbinom()` is doing: it is the *same function* whether you use it to compute a probability (fix `prob`, vary `x`) or a likelihood (fix `x`, vary `prob`). Only your reading of it changes.

2.2 Why we work in logs

[illegible]

```
sum(dbinom(y, size = 1, prob = 0.3, log = TRUE))
#> [1] -324.07
```

💡 Tip

Every `d*`() function in R takes `log = TRUE`. Use it. Computing `log(dnorm(x))` instead of `dnorm(x, log = TRUE)` is both slower and less accurate.

3 Maximizing: the Poisson case

The GSS variable `childs` counts a respondent's children. Start with the simplest possible model, $Y_i \sim \text{Poisson}(\theta)$ with no covariates.

```
y <- gss$childs

nll_pois <- function(theta) -sum(dpois(y, lambda = theta, log = TRUE))

fit <- optim(par = 1, fn = nll_pois, method = "Brent",
            lower = 0.01, upper = 20, hessian = TRUE)

c(mle_numeric = fit$par, closed_form = mean(y), converged = fit$convergence)
#> mle_numeric closed_form converged
#>      1.6044      1.6044      0.0000

th <- seq(1.2, 2.1, length.out = 300)
plot(th, -sapply(th, nll_pois), type = "l", col = "navy", lwd = 2,
     xlab = expression(theta), ylab = expression(ell(theta)))
abline(v = mean(y), col = "firebrick", lty = 2)
```

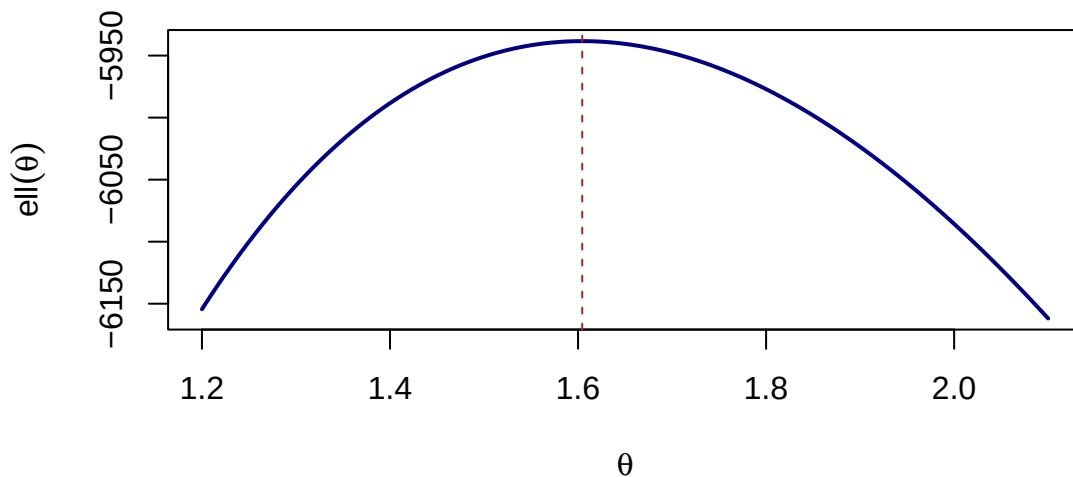


Figure 2: Poisson log-likelihood for the number of children, GSS.

3.1 Standard errors from the Hessian

We minimized $-\ell$, so the Hessian `optim()` returns is already $-\partial^2\ell/\partial\theta^2$ — the observed information. Its inverse is the variance.

```
se_hessian <- sqrt(1 / fit$hessian)
se_analytic <- sqrt(mean(y) / length(y)) # from I(theta) = n/theta

c(hessian = as.numeric(se_hessian), analytic = se_analytic)
#> hessian analytic
#> 0.021383 0.021383
```

They agree, as they must: the analytic result $\mathcal{I}(\theta) = n/\theta$ is what the numerical second derivative is approximating.

```
c(estimate = mean(y),
  lower = mean(y) - 1.96 * se_analytic,
  upper = mean(y) + 1.96 * se_analytic)
#> estimate lower upper
#> 1.6044 1.5625 1.6464
```

4 Two parameters: the normal likelihood

```
z <- gss$lnearn

nll_norm <- function(par) {
  mu <- par[1]; sigma <- exp(par[2]) # log-parameterize sigma to keep it > 0
  -sum(dnorm(z, mean = mu, sd = sigma, log = TRUE))
}

fit2 <- optim(c(10, 0), nll_norm, method = "BFGS", hessian = TRUE)

c(mu_hat = fit2$par[1], closed_form_mu = mean(z),
  sigma_hat = exp(fit2$par[2]),
  closed_form_sigma = sqrt(mean((z - mean(z))^2)))
#> mu_hat closed_form_mu sigma_hat closed_form_sigma
#> 9.94998 9.94999 0.93419 0.93418
```

i Note

Two things worth noticing.

1. We optimized over $\log \sigma$ rather than σ . Constrained parameters (variances, probabilities, rates) should be transformed to the whole real line before handing them to an unconstrained optimizer. By **invariance**, $\hat{\sigma} = \exp(\widehat{\log \sigma})$ is still the MLE.

2. The MLE of σ^2 divides by n , not $n - 1$. It is *biased*. Compare `sqrt(mean((z - mean(z))^2))` with `sd(z)`.

```
c(mle = sqrt(mean((z - mean(z))^2)), sd_function = sd(z))
#>      mle sd_function
#> 0.93418 0.93432
```

4.1 The likelihood surface

```
mu_grid <- seq(9.85, 10.05, length.out = 80)
sigma_grid <- seq(0.87, 1.00, length.out = 80)
g <- expand.grid(mu = mu_grid, sigma = sigma_grid)
g$ll <- mapply(function(m, s) sum(dnorm(z, m, s, log = TRUE)), g$mu, g$sigma)

ggplot(g, aes(mu, sigma, z = ll)) +
  geom_contour_filled(bins = 12) +
  geom_point(aes(x = mean(z), y = sqrt(mean((z - mean(z))^2))),
             colour = "white", size = 2, inherit.aes = FALSE) +
  labs(x = expression(mu), y = expression(sigma), fill = expression(ll)) +
  theme_minimal(base_size = 9) + theme(legend.key.size = unit(0.3, "cm"))
```

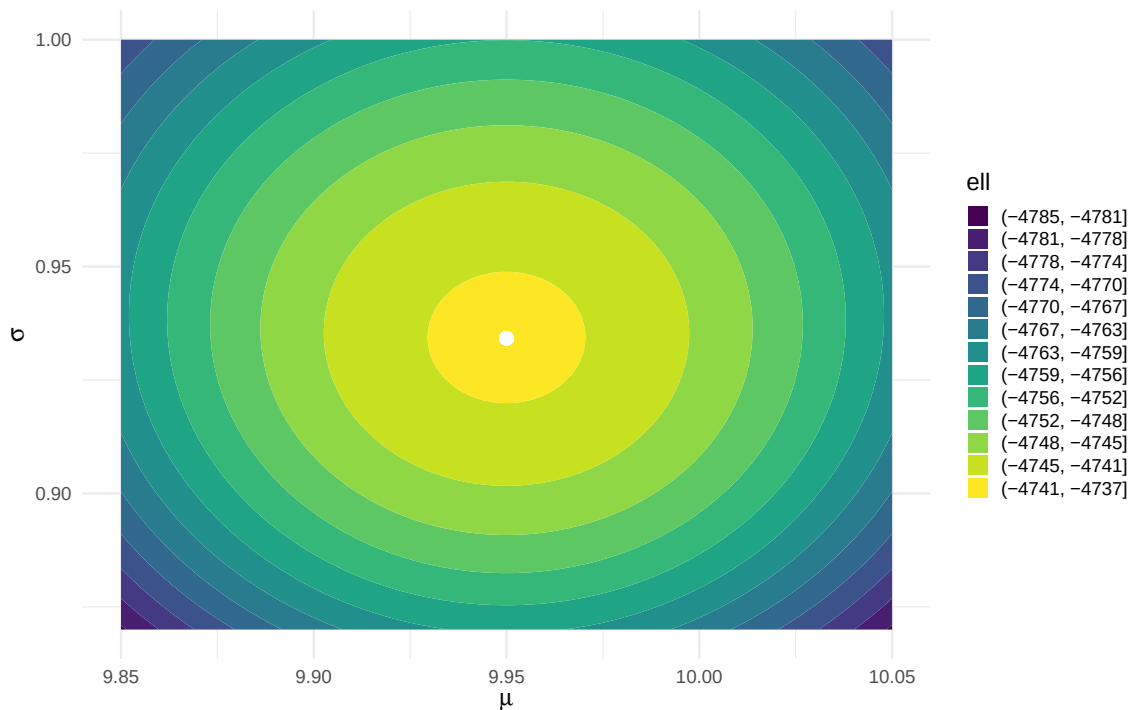


Figure 3: Log-likelihood surface for the normal model. The contours are ellipses centred on the MLE.

5 What non-identification looks like

Suppose we (foolishly) parameterize the mean as $\theta_1 + \theta_2$. The data can only speak to the sum.

```
set.seed(1)
w <- rnorm(200, mean = 3, sd = 1)

nll_bad <- function(par) -sum(dnorm(w, mean = par[1] + par[2], sd = 1, log = TRUE))

starts <- matrix(rnorm(60, sd = 3), ncol = 2)
res <- t(apply(starts, 1, function(s) optim(s, nll_bad)$par))
head(round(res, 3), 4)
#>      [,1] [,2]
#> [1,]  2.190 0.845
#> [2,]  6.200 -3.164
#> [3,]  4.525 -1.490
#> [4,] -0.548  3.584

plot(res, col = "navy", pch = 19, cex = .7,
      xlab = expression(hat(theta)[1]), ylab = expression(hat(theta)[2]))
abline(a = mean(w), b = -1, col = "firebrick", lwd = 1.5)
```

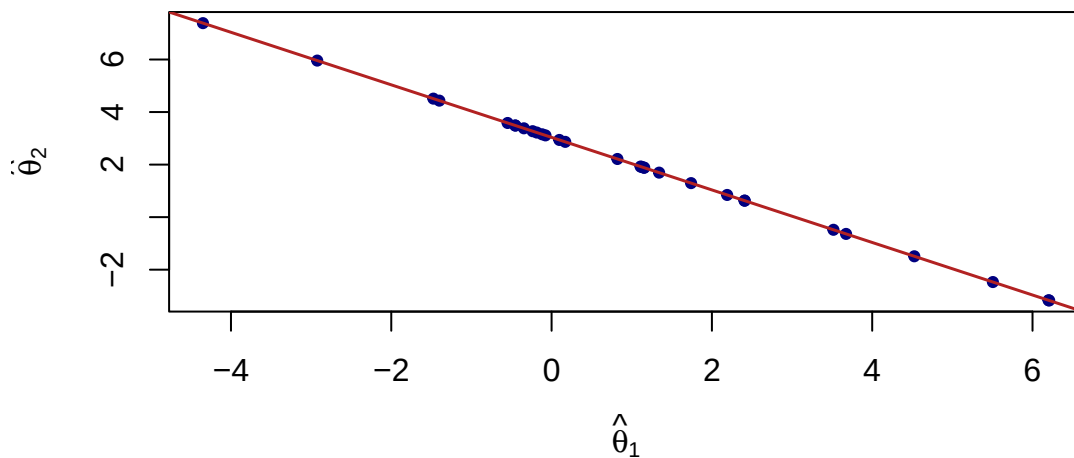


Figure 4: Estimates from 30 different starting values. They lie on the ridge $\theta_1 + \theta_2 = \bar{w}$.

The Hessian is singular, so there are no standard errors:

```
H <- optim(c(0, 0), nll_bad, hessian = TRUE)$hessian
H
#>      [,1] [,2]
#> [1,]  200  200
#> [2,]  200  200
qr(H)$rank      # 1, not 2
#> [1] 1
```

```
try(solve(H), silent = TRUE) |> class()
#> [1] "try-error"
```

! Important

This is the likelihood analogue of perfect collinearity. **The lesson:** always run your optimizer from several starting values. If they disagree, you have an identification problem, not a numerical one.

6 Logit from scratch

Now put covariates inside. Outcome: whether the respondent holds a bachelor's degree or more (ba).

```
X <- model.matrix(~ pareduc + wordsum + female + black, data = gss)
yb <- gss$ba

## log(1 + exp(eta)), computed without overflowing for large |eta|
log1pexp <- function(eta) ifelse(eta > 30, eta, log1p(exp(pmin(eta, 30))))

nll_logit <- function(beta) {
  eta <- as.vector(X %*% beta)
  -sum(yb * eta - log1pexp(eta))
}

## analytic gradient: -X'(y - p)
gr_logit <- function(beta) {
  p <- plogis(as.vector(X %*% beta))
  -as.vector(crossprod(X, yb - p))
}

fit_l <- optim(rep(0, ncol(X)), nll_logit, gr = gr_logit,
              method = "BFGS", hessian = TRUE)

m_glm <- glm(ba ~ pareduc + wordsum + female + black,
             data = gss, family = binomial(link = "logit"))

round(cbind(manual = fit_l$par, glm = coef(m_glm)), 5)
#>               manual          glm
#> (Intercept) -5.81041 -5.81041
#> pareduc      0.20656  0.20656
#> wordsum      0.39139  0.39139
#> female       0.27863  0.27863
```

```
#> black      -0.11440 -0.11440
```

Standard errors from the Hessian, versus what `glm()` reports:

```
round(cbind(manual = sqrt(diag(solve(fit_1$hessian))),
           glm = sqrt(diag(vcov(m_glm)))), 5)

#>               manual      glm
#> (Intercept) 0.24540 0.24539
#> pareduc     0.01342 0.01342
#> wordsum     0.02404 0.02404
#> female      0.07756 0.07756
#> black       0.10982 0.10982
```

Identical to four or five decimal places. `glm()` is doing exactly what we just did, only with Fisher scoring instead of BFGS.

```
## the score at the MLE should be zero
round(gr_logit(coef(m_glm)), 6)
#> [1] 0.000000 0.000001 0.000001 0.000000 0.000000
```

7 The three tests, by hand

Test H_0 : the coefficients on `wordsum` and `black` are both zero.

[illegible]

```
anova(m_rest, m_full, test = "Chisq")
#> Analysis of Deviance Table
#>
#> Model 1: ba ~ pareduc + female
#> Model 2: ba ~ pareduc + wordsum + female + black
#>   Resid. Df Resid. Dev Df Deviance      Pr(>Chi)
#> 1      3506      4282
```


[illegible]

All three point the same way, as the theory promises, and they differ a little in magnitude, as the theory also promises.

8 Exercises

1. **Bernoulli information.** Simulate $n = 200$ draws from `Bernoulli(0.3)`. Compute the MLE, the observed information from `optim(hessian = TRUE)`, and the analytic $\mathcal{I}(\theta) = n/[\theta(1 - \theta)]$ evaluated at $\hat{\theta}$. Confirm all three standard errors agree.
2. **Curvature is information.** For $n \in \{25, 100, 400, 1600\}$, simulate `Poisson(2)` data and plot the four log-likelihoods on the same axes, each shifted to peak at zero. Describe what happens to the width of the peak, and connect it to $\mathcal{I}_n = n\mathcal{I}_1$.
3. **A biased MLE.** Simulate $n = 50$ draws from `Uniform[0,  $\theta$ ]` with $\theta = 10$. The MLE is $\max_i Y_i$. Over 5,000 replications, compute the mean of the MLE. How large is the bias, and which regularity condition has failed?
4. **Probit from scratch.** Write the Bernoulli log-likelihood directly with $p_i = \Phi(X_i'\beta)$, i.e. `sum(yb * pnorm(eta, log.p = TRUE) + (1 - yb) * pnorm(-eta, log.p = TRUE))`, and reproduce `glm(..., family = binomial(link = "probit"))`. (Note you cannot get there by swapping `plogis` for `pnorm` in the *canonical-link* expression above — that simplification is specific to the logit.) Compare the two sets of coefficients: is the ratio close to 1.6?
5. **Invariance.** For the logit model above, the odds ratio for `wordsum` is $e^{\hat{\beta}}$. Get a standard error for it two ways: the delta method (`car::deltaMethod`) and by simulating 10,000 draws of $\tilde{\beta}$ from $\mathcal{N}(\hat{\beta}, \widehat{\text{Var}}(\hat{\beta}))$ and exponentiating. Which interval would you report, and why?
6. **A misspecified model.** Fit a Poisson model to `childs` and compare the model-based standard error with `sandwich::vcovHC()`. What does the gap tell you about the Poisson variance assumption? (This previews overdispersion, which is Week 4.)

9 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
```

```

#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] ggplot2_4.0.0 dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#> [1] vctrs_0.6.5      cli_3.6.5        knitr_1.48       rlang_1.3.0
#> [5] xfun_0.47        generics_0.1.3   S7_0.2.0         jsonlite_1.8.9
#> [9] labeling_0.4.3   glue_1.7.0       isoband_0.2.7    htmltools_0.5.8.1
#> [13] tinytex_0.53     scales_1.4.0     fansi_1.0.6      rmarkdown_2.28
#> [17] grid_4.4.1       evaluate_1.0.0   tibble_3.2.1     fastmap_1.2.0
#> [21] yaml_2.3.10      lifecycle_1.0.4  compiler_4.4.1   RColorBrewer_1.1-3
#> [25] pkgconfig_2.0.3  farver_2.1.2     digest_0.6.37    viridisLite_0.4.2
#> [29] R6_2.5.1         tidyselect_1.2.1 utf8_1.2.4       pillar_1.9.0
#> [33] magrittr_2.0.3   withr_3.0.1      tools_4.4.1      gtable_0.3.6

```